Part 1

Task 1.1
Relation A:
1) Superkeys: SSN+Name, Email+Name, Email+Name+Department, Phone+Name, Phone+Department, EmpID+Salary+Name
2) Candidate keys: EmpID, SSN, Email, Phone
3) EmpID, because it is natural key, not surrogate
4) No

Relation B:
1) StudentID, CourseCode, Section, Semester, Year
2) StudentID is needed because we need to know which student took the course. CourseCode is needed bacause we need to know which course is taken.
Section is needed because by the rule a student cannot take the same course section in the same semester.
Semester is needed because for each year we have 2 semester, and we need to identify them
Year is needed because students take courses each year
3) there is no

Task 1.2
enrollment.student ID to Student.Student ID
Student.advisorID to Professor.profID
`enrollment.course_id` to `course.course_id`
Course.departmentCode to department.deptCode
Professor.Department to department.dept_code

Part 2

Task 2.1
Strong: Patients, Doctors, Departments
Week: Appointments, Prescriptions, Hospital Rooms

Patients:
Patient IDS - key
names - simple
birthdates - simple
addresses - composite
phone numbers - multivalued
insurance information - simple/key

doctors:
Doctor ID - key
names - simple

specialization - multivalued
Phone number - simple.
Office location - simple.

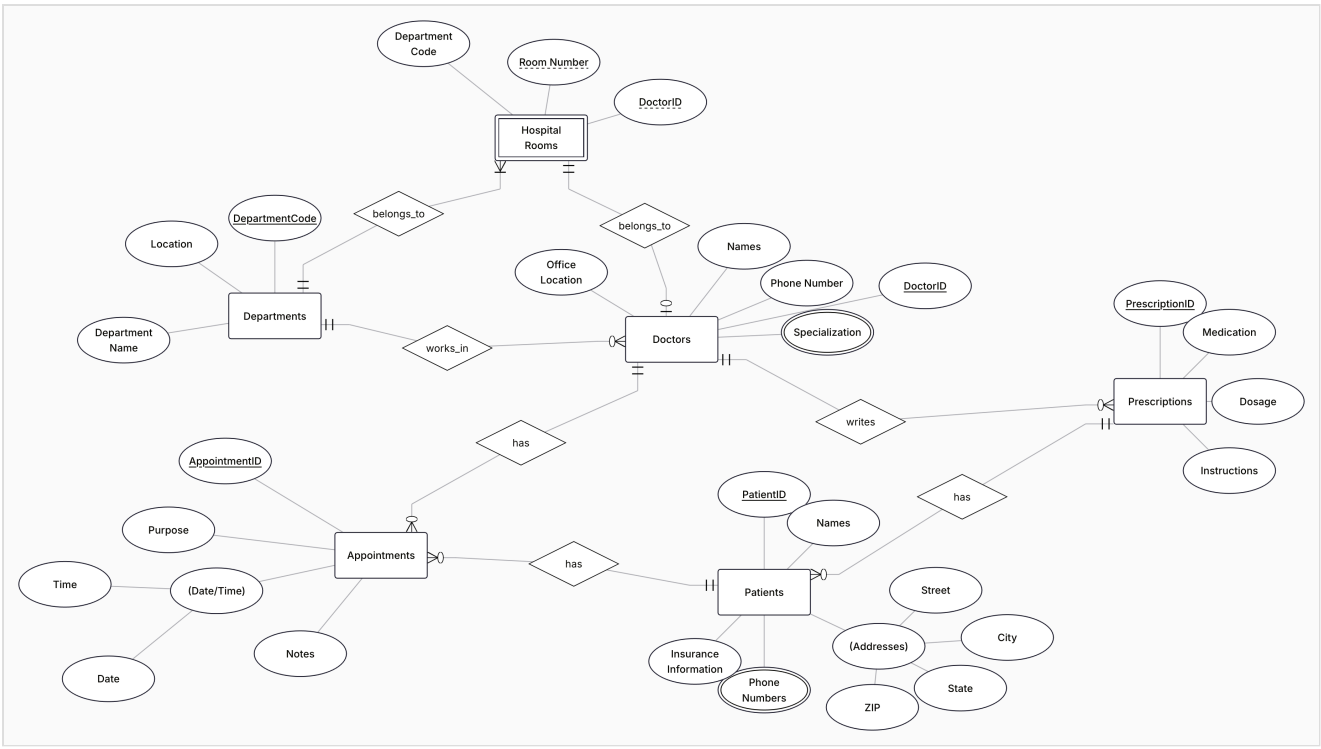
Departments:
department code - key
department name - simple
location - simple

Appointments:
appointment ID - key
date/time - composite
purpose - simple
notes - simple

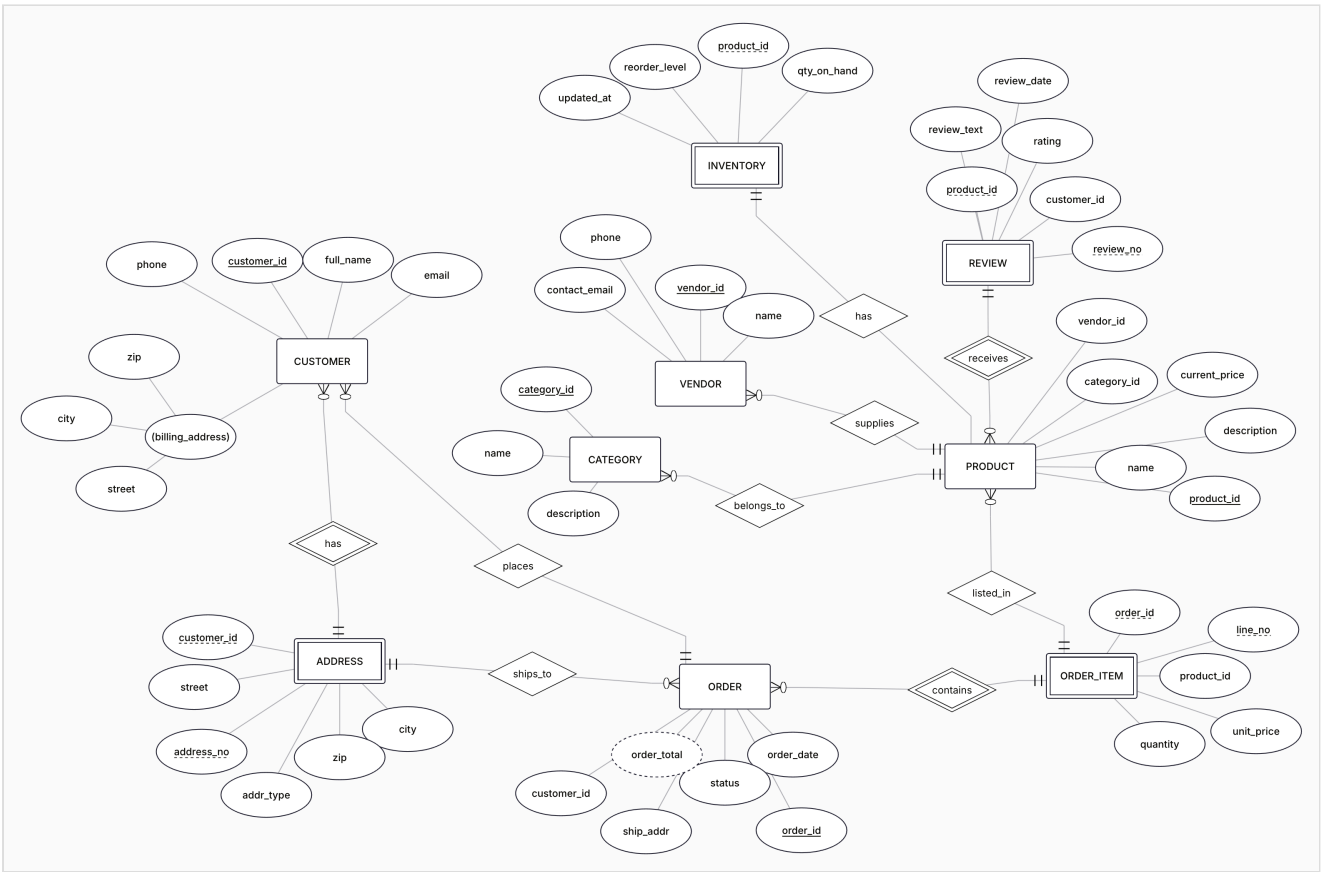
Prescriptions:
prescription ID - key
medication - simple
dosage - simple
instructions - simple

Hospital Rooms (weak entity):
room number - partial key
department code - key

Patients - Doctors: M:N
Patient -> Appointment: 1:N
Doctor -> Appointment: 1:N
Patient -> Prescriptions: 1:N
Doctor -> Prescriptions: 1:N
Departments - Hospitals: 1:N
Doctor - Hospital Rooms: N:1


Department Code
Room Number
DoctorID
Hospital Rooms
belongs_to
belongs_to
DepartmentCode
Location
Office Location
Names
Phone Number
DoctorID
Specialization
Departments
Department Name
works_in
Doctors
PrescriptionID
Medication
Prescriptions
Dosage
Instructions
writes
has
AppointmentID
Purpose
Time
(Date/Time)
Notes
Date
Appointments
has
PatientID
Names
Patients
has
Street
City
State
ZIP
(Addresses)
Insurance Information
Phone Numbers

Task 2.2


product_id
reorder_level
qty_on_hand
updated_at
INVENTORY
review_date
review_text
rating
product_id
customer_id
REVIEW
review_no
phone
customer_id
full_name
email
CUSTOMER
zip
city
(billing_address)
street
has
phone
contact_email
vendor_id
name
VENDOR
has
vendor_id
receives
category_id
name
CATEGORY
description
belongs_to
supplies
category_id
current_price
description
name
product_id
PRODUCT
has
places
listed_in
customer_id
street
address_no
addr_type
zip
city
ADDRESS
ships_to
ORDER
order_total
order_date
status
customer_id
ship_addr
order_id
contains
order_id
line_no
product_id
quantity
unit_price
ORDER_ITEM

Part 4

Task 4.1

1)
FDs:
StudentID -> StudentName, StudentMajor
ProjectID -> ProjectTitle, ProjectType, SupervisorID
SupervisorID -> SupervisorName, SupervisorDept
{StudentID, ProjectID} -> Role, HoursWorked, StartDate, EndDate

2)
Redundancy: StudentName is repeated in every row of the student, ProjectTitle - in every row of the project, SupervisorName - in all projects of this supervisor.
Update: if we change major of the student, we need to update N rows, if we miss one - data will be wrong.
Insert: we cannot insert new student without project, and new supervisor without project too.
Delete: if we delete last project of the student - we lose the student and the supervisor too.

3)
There is no 1NF violations

4)
StudentID and ProjectID are partial dependecies

Here is the decomposition:

Student(StudentID PK, StudentName, StudentMajor)
Project(ProjectID PK, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept)
Assignment(StudentID FK, ProjectID FK, Role, HoursWorked, StartDate, EndDate)

5)

Here we have transitive dependencies

ProjectID -> SupervisorID -> SupervisorName, SupervisorDept

so we need to separate the supervisor table

Student(StudentID PK, StudentName, StudentMajor)
Supervisor(SupervisorID PK, SupervisorName, SupervisorDept)
Project(ProjectID PK, ProjectTitle, ProjectType, SupervisorID FK -> Supervisor)
Assignment(StudentID FK -> Student, ProjectID FK -> Project, Role, HoursWorked, StartDate, EndDate)

Task 4.2

1)
PK = StudentID + TimeSlot + Room

2)

StudentID -> StudentMajor
CourseID -> CourseName
InstructorID -> InstructorName
Room -> Building
{TimeSlot, Room} -> CourseID, InstructorID
{InstructorID, TimeSlot} -> Room, CourseID

3)
Not in BCNF.
{TimeSlot, Room} -> CourseID, InstructorID, but {TimeSlot, Room} alone is not a superkey (needs StudentID). Same for {InstructorID, TimeSlot} -> Room,

4)
Decomposition to BCNF (lossless, on PK=FK):

Student(StudentID PK, StudentMajor)
Course(CourseID PK, CourseName)
Instructor(InstructorID PK, InstructorName)
RoomLoc(Room PK, Building)
Section(TimeSlot, Room, CourseID FK -> Course, InstructorID FK -> Instructor)
PK = (TimeSlot, Room)
UNIQUE(InstructorID, TimeSlot)
Enrollment(StudentID FK -> Student, TimeSlot, Room FK -> Section)
PK = (StudentID, TimeSlot, Room)

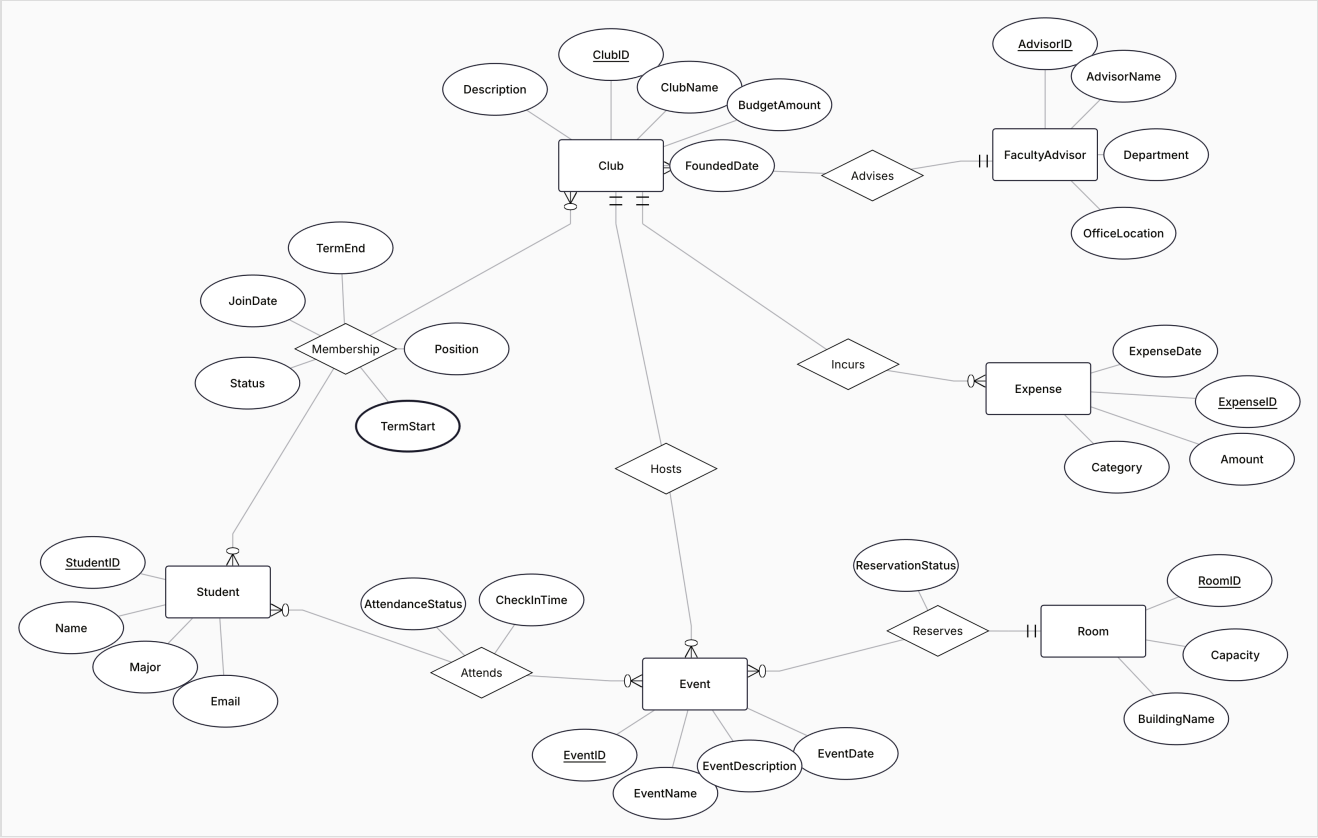
Each determinant is now a key: {TimeSlot, Room} is PK of Section, Room is PK of RoomLoc, etc.

5)
No data loss if joined on keys (lossless join). But dependency preservation is lost for constraints spanning two tables: e.g. {StudentID, CourseID, TimeSlot} -> Room now needs Enrollment JOIN Section and cannot be enforced by a single PRIMARY KEY / UNIQUE. In Postgres it would need a trigger/assertion, not just a constraint. Queries also become joins instead of single-table scans.

Part 5

Task 5.1

1)
ER diagram: https://erdplus.com/diagrams/588649



Entities: Student, Club, FacultyAdvisor, Event, Room, Expense
Relationships:
- Membership: Student M:N Club (JoinDate, Status, Position, TermStart, TermEnd)
- Advises: FacultyAdvisor (1) -- (N) Club, each Club mandatory/one advisor
- Hosts: Club (1) -- (N) Event, each Event mandatory/one club
- Attends: Student M:N Event (AttendanceStatus, CheckInTime)
- Reserves: Event (N) -- (1) Room (ReservationStatus), each Event mandatory/one room
- Incurs: Club (1) -- (N) Expense, each Expense mandatory/one club

2)
Relational schema (PK underlined, FK noted):

Student(StudentID PK, Name, Email, Major)
FacultyAdvisor(AdvisorID PK, AdvisorName, Department, OfficeLocation)
Club(ClubID PK, ClubName, FoundedDate, Description, BudgetAmount, AdvisorID FK -> FacultyAdvisor)
Room(RoomID PK, BuildingName, Capacity)
Event(EventID PK, EventName, EventDate, EventDescription, ClubID FK -> Club, RoomID FK -> Room, ReservationStatus)
Membership(StudentID FK -> Student, ClubID FK -> Club, JoinDate, Status, Position, TermStart, TermEnd)
PK = (StudentID, ClubID)
Attendance(StudentID FK -> Student, EventID FK -> Event, AttendanceStatus, CheckInTime)
PK = (StudentID, EventID)
Expense(ExpenseID PK, Amount, ExpenseDate, Category, ClubID FK -> Club)

3)
Design decision with multiple valid options: officer positions.

Option A — add Position/TermStart/TermEnd columns directly to Membership (chosen): fewer tables, one row per student-club pair covers both membership and role, and "is this student an officer" is just `Position IS NOT NULL`.
Option B — a separate Officer table/relationship, keyed by (StudentID, ClubID, TermStart).

Why A: simpler schema, no extra join for the common case, no need to keep Officer's foreign key in sync with Membership by hand. Tradeoff: since Membership's PK is (StudentID, ClubID), a student can only hold one officer term per club at a time in this schema — electing a new president overwrites the old TermEnd/Position instead of preserving history of past terms. That's acceptable here because the club-tracking system only needs "who is the current officer," not a historical audit of past terms; if term history mattered, Option B (or a separate OfficerHistory table) would be necessary.

4)
Example queries (English):
- "Find all students who are officers in the Computer Science Club"
- "List all events scheduled for next week with their room reservations"
- "Show total expenses this semester for each club compared to its allocated budget"